

Ordered Digit Operations in C

Algorithms Test Cases Program and Execution

Name Reyansh Pandit

Batch 16

SAP ID 590033602

1 Problem and interpretation

This menu-driven program performs four digit-based operations. Every operation is implemented as a separate void function with no parameters; each function reads its own positive integer, processes it, and displays the result. Increasing and decreasing mean strict left-to-right order. Second largest and second smallest mean the second distinct digits.

2 Algorithms

Part	Algorithm
Main menu	1. Display choices 1 to 5. 2. Read choice. 3. Use switch-case to make a simple call to the selected void function. 4. Repeat with do-while until choice 5.
Increasing order	1. Read and validate a positive integer. 2. Extract digits from right to left. 3. Each new digit must be smaller than the digit previously extracted. 4. If not, report not increasing; otherwise report increasing.
Decreasing order	1. Read and validate a positive integer. 2. Extract digits from right to left. 3. Each new digit must be larger than the digit previously extracted. 4. If not, report not decreasing; otherwise report decreasing.
Largest and smallest	1. Read and validate the number. 2. Initialize largest and smallest with its last digit. 3. Extract remaining digits and update both values. 4. Display both.
Second largest and smallest	1. Read and validate the number. 2. Track largest, second largest, smallest, and second smallest without arrays. 3. Ignore repeated values when finding second positions. 4. Display results or state that a second distinct digit does not exist.

3 Test case table

Category	Input	Expected	Observed	Status
Invalid input	choice 1 then -5, 0, 123	Reject -5 and 0; accept 123	As expected	Pass
Increasing true	12359	Increasing order	Increasing order	Pass
Increasing false	1224	Not increasing	Not increasing	Pass
Increasing with zero	120	Not increasing	Not increasing	Pass
Single digit	7	Increasing and decreasing	As expected	Pass
Decreasing true	97520	Decreasing order	Decreasing order	Pass
Decreasing false	97525	Not decreasing	Not decreasing	Pass
Largest smallest	50291	Largest 9; smallest 0	9 and 0	Pass
Repeated extremes	99011	Largest 9; smallest 1	9 and 1	Pass
Second digits	77420	Second largest 4; second smallest 2	4 and 2	Pass
Two distinct digits	5552	Second largest 2; second smallest 5	2 and 5	Pass
No second distinct	777	Two no-second messages	As expected	Pass
Invalid menu	choice 6	Invalid choice	Invalid choice	Pass

4 C program

```
#include <stdio.h>

void check_increasing(void);
void check_decreasing(void);
void find_largest_smallest(void);
void find_second_largest_smallest(void);

void check_increasing(void)
{
    int number, digit, previous = -1;
    int increasing = 1;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    for (; number != 0; number /= 10) {
        digit = number % 10;
        if (previous != -1 && digit >= previous) {
            increasing = 0;
            break;
        }
        previous = digit;
    }

    if (increasing)
        printf("Digits are in increasing order.\n");
    else
        printf("Digits are not in increasing order.\n");
}

void check_decreasing(void)
{
    int number, digit, previous = -1;
    int decreasing = 1;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    for (; number != 0; number /= 10) {
        digit = number % 10;
        if (previous != -1 && digit <= previous) {
            decreasing = 0;
            break;
        }
        previous = digit;
    }

    if (decreasing)
        printf("Digits are in decreasing order.\n");
    else
        printf("Digits are not in decreasing order.\n");
}

void find_largest_smallest(void)
{
    int number, digit, largest, smallest;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    largest = smallest = number % 10;
    for (number /= 10; number != 0; number /= 10) {
        digit = number % 10;
        if (digit > largest)
            largest = digit;
        if (digit < smallest)
            smallest = digit;
    }
}
```

```

    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void find_second_largest_smallest(void)
{
    int number, digit;
    int largest = -1, second_largest = -1;
    int smallest = 10, second_smallest = 10;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    for (; number != 0; number /= 10) {
        digit = number % 10;

        if (digit > largest) {
            second_largest = largest;
            largest = digit;
        } else if (digit != largest && digit > second_largest) {
            second_largest = digit;
        }

        if (digit < smallest) {
            second_smallest = smallest;
            smallest = digit;
        } else if (digit != smallest && digit < second_smallest) {
            second_smallest = digit;
        }
    }

    if (second_largest == -1)
        printf("No second largest distinct digit.\n");
    else
        printf("Second largest digit = %d\n", second_largest);

    if (second_smallest == 10)
        printf("No second smallest distinct digit.\n");
    else
        printf("Second smallest digit = %d\n", second_smallest);
}

int main(void)
{
    int choice;

    do {
        printf("\n1. Check increasing digit order\n");
        printf("2. Check decreasing digit order\n");
        printf("3. Find largest and smallest digits\n");
        printf("4. Find second largest and second smallest digits\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: check_increasing(); break;
            case 2: check_decreasing(); break;
            case 3: find_largest_smallest(); break;
            case 4: find_second_largest_smallest(); break;
            case 5: printf("Program terminated.\n"); break;
            default: printf("Invalid choice. Select 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}

```

5 Compilation

```
gcc -std=c11 -Wall -Wextra -pedantic ordered_digit_operations.c -o ordered_digit_operations
```

Compilation completed successfully with no warnings or errors.

6 Execution

```
(kali[kali])- [~/demo1]
$ gcc -std=c11 -Wall -Wextra -pedantic ordered_digit_operations.c -o ordered_digit_operations
(kali[kali])- [~/demo1]
$ ./ordered_digit_operations

1. Check increasing digit order
2. Check decreasing digit order
3. Find largest and smallest digits
4. Find second largest and second smallest digits
5. Exit
Enter your choice: 1
Enter a positive integer: 12359
Digits are in increasing order.

Enter your choice: 2
Enter a positive integer: 97520
Digits are in decreasing order.

Enter your choice: 3
Enter a positive integer: 50291
Largest digit = 9
Smallest digit = 0

Enter your choice: 4
Enter a positive integer: 77420
Second largest digit = 4
Second smallest digit = 2

Enter your choice: 5
Program terminated.
```

Figure 1 Compilation and execution of all four operations

7 Test result

The compiled program was executed against all prepared valid, invalid, boundary, repeated-digit, and insufficient-distinct-digit cases. Every observed result matched the expected result.

8 Conclusion

The program meets the requirement for a switch-case menu and separate parameterless void functions. It correctly checks strict digit ordering and finds the largest, smallest, second largest, and second smallest distinct digits.